



Memory Management

Operating Systems

Universität Innsbruck

Summer Semester 2026

1 Real Linux Memory Observation

1.1 Process Memory Layout

In this exercise, you will observe the virtual memory layout of a Linux process and relate program variables to different memory regions such as the text segment, data segment, heap, stack, and shared libraries. You will also inspect how Linux maps process memory using `/proc/<pid>/maps` and `pmap`.

1.1.1 Step 1: Execute the Program

Consider the following program and execute it:

`01_memory_layout.c`

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int global_var = 10;
5
6 int main() {
7     int local_var = 20;
8
9     void *heap_ptr = malloc(4096);
10
11     printf("global: %p\n", &global_var);
12     printf("local : %p\n", &local_var);
13     printf("heap  : %p\n", heap_ptr);
14
15     getchar();
16     return 0;
17 }
```

Example output:

```
global: 0x561d1cb2b030
local : 0x7ffe2b87d8fc
heap  : 0x561d1df3c2a0
```

1.1.2 Step 2: Find the Process ID

In another terminal, run:

```
ps -ef | grep 01_memory_layout
```

1.1.3 Step 3: Inspect Process Memory Mappings

Run:

```
cat /proc/pid/maps
```

or:

```
pmap pid
```

In `/proc/pid/maps`, each line represents one virtual memory region. The general format is as follows:

```
start-end permissions offset device inode pathname
```

Example:

```
7ffe2b85f000-7ffe2b880000 rw-p ... [stack]
```

pmap displays the memory mappings of a process in a simplified and human-readable form. Example output can be:

```

1 25431: ./01_memory_layout
2 000055b8d8f8c000 4K r----- 01_memory_layout
3 000055b8d8f8d000 4K r-x-- 01_memory_layout
4 000055b8d8f8e000 4K r----- 01_memory_layout
5 000055b8d8f8f000 4K rw--- 01_memory_layout
6 000055b8da9b7000 132K rw--- [ heap ]
7 00007f4d9d000000 1048576K rw--- [ anon ]
8 00007f4ddeb00000 1900K r-x-- libc.so.6
9 00007ffe7a6e5000 132K rw--- [ stack ] total 1050756K

```

1.1.4 Step 4: Check Address Space Layout Randomization (ASLR)

Run the program multiple times. You will notice that addresses change between executions. This is caused by ASLR. Linux randomizes memory layout locations to make exploitation attacks more difficult.

1.1.5 Step 5: Identify Read-Only vs Writable Regions

Observe permission bits in `/proc/<pid>/maps`.

- For example, **r-xp** means readable, executable, and not writable. It is typically used for executable code.
- For example, **rw-p** means readable, writable, and not executable. It is typically used for heap, stack, and writable data.

1.2 Lazy Allocation and RSS vs Virtual Memory

In this exercise, we demonstrate one of the most important concepts in modern operating systems: **virtual memory != physical memory**. A process may reserve a very large virtual address space without immediately consuming the same amount of physical RAM.

1.2.1 Step 1: Run The Program

Execute the following program:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main() {
6     size_t size = 1024L * 1024L * 1024L;
7
8     char *p = malloc(size);
9
10    printf("Allocated 1 GB\n");
11    getchar();
12
13    for (size_t i = 0; i < size; i += 4096) {
14        p[i] = 1;
15    }
16
17    printf("Touched all pages\n");
18    getchar();
19
20    free(p);
21 }
```

The program prints:

Allocated 1 GB

and pauses at `getchar()`. Do not press Enter yet.

1.2.2 Step 2: Observe Memory Usage and Page Fault Before Touching Pages

In another terminal, inspect the process using `top` or `htop` and `pmap <pid>` and `cat /proc/<pid>/status`. In `/proc/<pid>/status`, observe `VmSize` and `VmRSS`.

Meaning of VmSize VmSize represents the total virtual memory size reserved by the process. This includes allocated heap memory, shared libraries, stack, and mapped regions. After `malloc(1 GB)`, VmSize should increase significantly.

Meaning of VmRSS VmRSS stands for Resident Set Size. It represents the amount of physical RAM currently occupied by the process. This is an important observation: Even though the program allocated 1 GB, the RSS remains relatively small.

Why RSS Is Small? The call `malloc(1 GB)` does NOT immediately allocate 1 GB of physical RAM. Instead:

- Linux reserves virtual address space;
- actual physical pages are allocated only when accessed.

This mechanism is called **lazy allocation** or **demand paging**.

Page Fault Counters Now we want to check the total number of page faults at this stage. You can see the content of `stat` file with:

```
cat /proc/<pid>/stat
```

or more conveniently with:

```
awk '{print "Minor faults:", $10, "Major faults:", $12}' \
/proc/<pid>/stat
```

`awk` processes text files column by column.

It will give an output like:

```
Minor faults: 86
Major faults: 0
```

Minor page fault means the page was not mapped, but no disk I/O was needed, such as heap growth, stack growth and copy-on-Write after `fork()`.

Major page fault means the page had to be loaded from disk, such as a large file mapped with `mmap()`.

1.2.3 Step 3: Touch the Pages

Return to the program and press Enter. The loop now executes:

```
1 for (size_t i = 0; i < size; i += 4096) {
2     p[i] = 1;
3 }
```

Notice the step size: 4096 bytes, which is typically the page size on Linux systems. The program touches one byte in every page.

Why this matters? Before touching memory pages existed only as virtual mappings. After touching memory:

- the CPU accesses the pages;
- page faults occur;
- Linux allocates real physical frames;
- RSS increases significantly.

1.2.4 Step 4: Observe RSS and Page Fault Counters Again

Inspect again using `top`, `htop`, `pmap <pid>` and `cat /proc/<pid>/status`. Now observe that VmRSS has increased dramatically. This shows that physical RAM is allocated only after the pages are accessed.

Now check minor and major page faults again. You will get an output like:

```
Minor faults: 262320
Major faults: 0
```

The difference in minor page faults is around 262,240. With the page size 4096 bytes and allocated memory of 1GB we have the number of pages allocated as:

$$1,073,741,824/4096 = 262,144pages$$

So touching every page once causes approximately 262,144 minor page faults, which matches the observation.

Additional Experiment: Try changing:

```
1 i += 4096
```

to:

```
1 i += 8192
```

Observe how memory usage and runtime behavior change. You can see that now about half the pages are loaded into memory (see the VmRSS value), due to accessing half of them.

1.3 Checking Page Fault Counters with perf stat Command

To check the page-fault value for a process, you can also use the `perf stat` command. Consider the following program:

`03_page_fault.c`

```
1 #include <stdlib.h>
2
3 int main() {
4     const long N = 1024L * 1024L * 1024L;
5
6     char *p = malloc(N);
7
8     for (long i = 0; i < N; i += 4096)
9         p[i]++;
10
11     free(p);
12 }
```

Now execute this command in the terminal:

```
perf stat ./03_page_fault
```

You can check the number of page faults in the output, such as:

```
524,341 page-faults
```

Recall:

1 GB / 4096 bytes/page = 262,144 pages

If you touch every page once we have:

262,144 first accesses → 262,144 page faults

The number 524,341 is roughly:

$2 \times 262,144$

which is quite plausible because:

- program startup causes faults
- libc causes faults
- stack growth causes faults
- some pages may be faulted multiple times during setup

Important to enable `perf` command Due to Linux security restrictions, the `perf stat` command most probably will raise a permission error, and you need to give access for performance monitoring. For temporary permission, use this command:

```
sudo sh -c 'echo -1 > /proc/sys/kernel/perf_event_paranoid'
```

For permanent permission add this line to `/etc/sysctl.conf` file:

```
kernel.perf_event_paranoid = -1
```

and then execute this command:

```
sudo sysctl -p
```

1.4 mmap() Shared Memory Between Processes

In this exercise, you will use shared memory between the parent and child processes and inspect it in their memory mapping. Consider the following program: [06_shm.c](#)

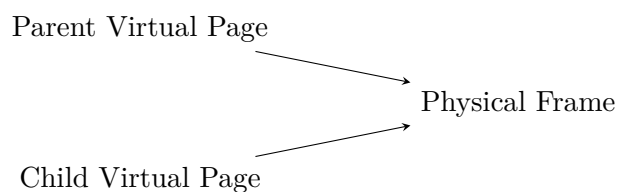
```
1 #define _GNU_SOURCE
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5 #include <sys/mman.h>
6 #include <sys/wait.h>
7 #define PAGE_SIZE 4096
8
9 int main(void) {
10     int *shared = mmap(NULL, PAGE_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED | ↵
        MAP_ANONYMOUS, -1, 0);
```

```

11
12  if (shared == MAP_FAILED) {
13      perror("mmap");
14      return 1;
15  }
16
17  shared[0] = 0;
18  printf("Parent PID: %d\n", getpid());
19  printf("Shared mapping address: %p\n", (void *)shared);
20  printf("Press Enter to fork...\n");
21  getchar();
22  pid_t pid = fork();
23
24  if (pid < 0) {
25      perror("fork");
26      return 1;
27  }
28
29  if (pid == 0) {
30      printf("\n[Child]\n");
31      printf("PID: %d\n", getpid());
32      printf("Shared address: %p\n", (void *)shared);
33      sleep(2);
34      printf("Value observed by child: %d\n", shared[0]);
35      printf("Press Enter to exit child process...\n");
36      getchar();
37      munmap(shared, PAGE_SIZE);
38      return 0; }
39  else {
40      printf("\n[Parent]\n");
41      printf("PID: %d\n", getpid());
42      printf("Child PID: %d\n", pid);
43      shared[0] = 123;
44      printf("Parent wrote shared[0] = 123\n");
45      wait(NULL);
46      munmap(shared, PAGE_SIZE);
47      return 0;
48  }
49 }

```

Here, multiple virtual pages point to the same physical frame. This is exactly how shared memory, shared libraries and memory-mapped files are implemented. Here, we have:



Both page tables point to the same frame.

After executing this program, you will get an output like:

Parent PID: 24510

```
Shared mapping address: 0x7f5d3e7f0000  
Press Enter to fork...
```

In another terminal:

```
cat /proc/24510/maps
```

or:

```
pmap 24510
```

You will see a memory region corresponding to the anonymous mapping. Example:

```
7f5d3e7f0000-7f5d3e801000 rw-s 00000000 00:01 ...
```

Notice rw-s, where:

r = readable
w = writable
s = shared

Now press Enter. The process now executes `fork()`. Parent and child continue. Before exiting the child process, check the maps file for it; you can also see its shared memory. This is the memory where both processes have access to it in the physical memory, and changes remain visible to both processes.

2 TLB Hits and Misses

The Translation Lookaside Buffer (TLB) is a small hardware cache inside the CPU that stores recently used virtual-to-physical address translations. During memory access, the CPU first checks whether the required page translation is already present in the TLB.

- **TLB Hit:** The translation is found in the TLB. The CPU can immediately obtain the physical address without consulting the page table.
- **TLB Miss:** The translation is not found in the TLB. The CPU must perform a page-table lookup to obtain the translation and may subsequently insert the result into the TLB.

Because page-table walks are significantly more expensive than TLB accesses, a high TLB hit rate is essential for achieving good memory performance.

Linux can collect TLB statistics using the `perf` utility. Execute the program `03_page_fault.c` and measure the number of TLB accesses and TLB misses using:

```
perf stat -e dTLB-loads,dTLB-load-misses ./bin/03_page_fault
```

You will get an output like:

```

397,094,262      dTLB-loads
724,799         dTLB-load-misses      #    0.18\% of all dTLB cache accesses

0.590714665 seconds time elapsed

0.116534000 seconds user
0.450063000 seconds sys

```

In this example:

- **dTLB-loads** represents the number of data-memory accesses that required a virtual-to-physical address translation.
- **dTLB-load-misses** represents the number of accesses for which the translation was not found in the TLB.
- The miss rate is reported as:

$$\text{TLB Miss Rate} = \frac{\text{dTLB-load-misses}}{\text{dTLB-loads}} \times 100$$

For the output above:

$$\frac{724,799}{397,094,262} \times 100 \approx 0.18\%$$

which indicates that more than 99.8% of address translations were served directly from the TLB.

2.1 Locality vs No Locality Experiment

A low TLB miss rate is typically achieved when memory accesses exhibit good spatial and temporal locality. Sequential array traversals, for example, often result in very few TLB misses because neighboring memory locations usually belong to the same page. In contrast, random accesses over a large memory region tend to increase the TLB miss rate because the working set may exceed the capacity of the TLB.

Now we want to compare the execution time of a program with accessing process pages, considering locality and non-locality. Consider the two programs: [04_tlb_good_locality.c](#)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define N 10000000
5 #define ITER 50
6
7 int main() {
8     int *array = malloc((size_t)N * sizeof(int));
9     if (array == NULL) {
10         perror("malloc");
11         return 1;

```

```
12     }
13
14     for (int i = 0; i < N; i++) {
15         array[i] = i % 100;
16     }
17
18     volatile long long sum = 0;
19
20     for (int iter = 0; iter < 1000; iter++)
21         for (int i = 0; i < 10000; i++)
22             sum += array[i];
23
24     free(array);
25     return 0;
26 }
```

05_tlb_poor_locality.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define N 10000000
5 #define ITER 50
6
7 int main() {
8     int *array = malloc((size_t)N * sizeof(int));
9     if (array == NULL) {
10         perror("malloc");
11         return 1;
12     }
13
14     for (int i = 0; i < N; i++) {
15         array[i] = i % 100;
16     }
17
18     volatile long long sum = 0;
19
20     for (int iter = 0; iter < 1000; iter++) {
21         for (int i = 0; i < 10000; i++) {
22             sum += array[(i * 7919) % 10000];
23         }
24     }
25
26     free(array);
27     return 0;
28 }
```

Now compare the execution times with these commands:

```
time ./04_tlb_good_locality
time ./05_tlb_poor_locality
```

Also check TLB/cache-related counters with these commands:

```
perf stat -e cache-misses,cache-references,dTLB-loads,dTLB-load-misses ./04_tlb_good_  
perf stat -e cache-misses,cache-references,dTLB-loads,dTLB-load-misses ./05_tlb_poor_
```

`05_tlb_good_locality` example accesses memory sequentially:

```
array[0], array[1], array[2], ...
```

This gives good spatial locality. CPU caches and TLB entries are reused efficiently.

`05_tlb_poor_locality` jumps around:

```
array[(i * 7919) % N]
```

This destroys locality. It causes more cache misses and usually more TLB misses. This is why memory access pattern matters, not only the number of operations.